

Gravitational Waves from Inspiralling Compact Binaries: Equation Summary

Aaryamann Kaushik

Overview

These are the equations behind two things: a simulation of a binary black hole inspiral, and a matched filter that pulls a weak gravitational-wave signal out of noisy data.

Orbital Dynamics

Velocity Evolution

The orbital velocity parameter grows over time as

$$\frac{dv}{dt} = \frac{32}{5} \left(\frac{\mu}{m^2} \right) v^9 \quad (1)$$

- $v(t)$: orbital velocity parameter (tied to orbital frequency)
- $m = m_1 + m_2$: total mass of the binary
- $\mu = \frac{m_1 m_2}{m}$: reduced mass

Once v hits 0.5, the evolution is cut off, since the post-Newtonian approximation used here stops being valid past that point. Both dv/dt and $d\phi/dt$ are set to zero from there on.

Orbital Phase

$$\frac{d\phi}{dt} = \frac{v^3}{m} \quad (2)$$

$\phi(t)$ is the orbital phase, in radians.

Gravitational-Wave Polarizations

$$h_+(t) = 4 \cdot \frac{\mu}{m} \cdot v^2(t) \cdot \cos(\phi(t)) \quad (3)$$

$$h_\times(t) = 4 \cdot \frac{\mu}{m} \cdot v^2(t) \cdot \sin(\phi(t)) \quad (4)$$

$h_+(t)$ and $h_\times(t)$ are the plus and cross polarizations of the gravitational wave.

Initial Conditions

- $v(t = 0) = v_0 = 0.3$
- $\phi(t = 0) = 0$
- $m_1 = 10 M_\odot, m_2 = 5 M_\odot$
- Integrated from $t = 0$ to $t = 19800$, in units of GM/c^3

Numerical Methods

This system gets solved two different ways: once with `scipy.integrate.odeint`, which takes fixed steps, and once with `scipy.integrate.ode` using the `dopri5` integrator, an adaptive-step Dormand-Prince Runge-Kutta method. Running both is a check that the fixed-step version isn't just quietly wrong because of a bad choice of step size.

Matched Filtering

Matched filtering finds a weak, known signal hiding in Gaussian noise. The data is sampled at discrete times t_i , spaced Δt apart. Given a template $h_i = h(t_i)$ and noisy data $x_i = h_i + n_i$, the template gets normalized by the noise level:

$$\hat{h}_i = \frac{h_i}{\sqrt{\Delta t \sum_i \frac{h_i^2}{\sigma^2}}} \quad (5)$$

Here σ is the standard deviation of the Gaussian noise.

The normalized template is then cross-correlated with the data at every possible shift index k :

$$R_k = \Delta t \sum_i x_i \hat{h}_{i-k} \quad (6)$$

This is done with `scipy.signal.correlate`. Wherever R_k peaks is the detection: the peak height is the signal-to-noise ratio, and the shift index it occurs at, converted to a time $\tau = k \Delta t$, tells you where in the data the signal sits.